CS7GV3 – Real-Time Rendering
Assignment 5: Moment Shadow Maps

Priyansh Nayak
Student ID: 25350660

Abstract

This report describes the implementation of hard shadow mapping, percentage-closer filtering (PCF), and Moment Shadow Mapping (MSM) in a reusable OpenGL engine. The work began with a directional depth shadow map, then added PCF, and finally extended the renderer with a four-moment MSM pipeline based on the 2015 paper by Peters and Klein [11]. To make the MSM result usable, I added stabilisation steps such as better bias handling, signed depth, clamping, and Gaussian filtering of the moment texture.

The final renderer can switch between Hard, PCF, and MSM on the same scene, which made it possible to compare the three methods directly in terms of shadow quality, runtime cost, and memory usage. MSM produced the smoothest and most natural-looking shadow transition, but at a noticeably higher cost than the baseline depth-based modes.

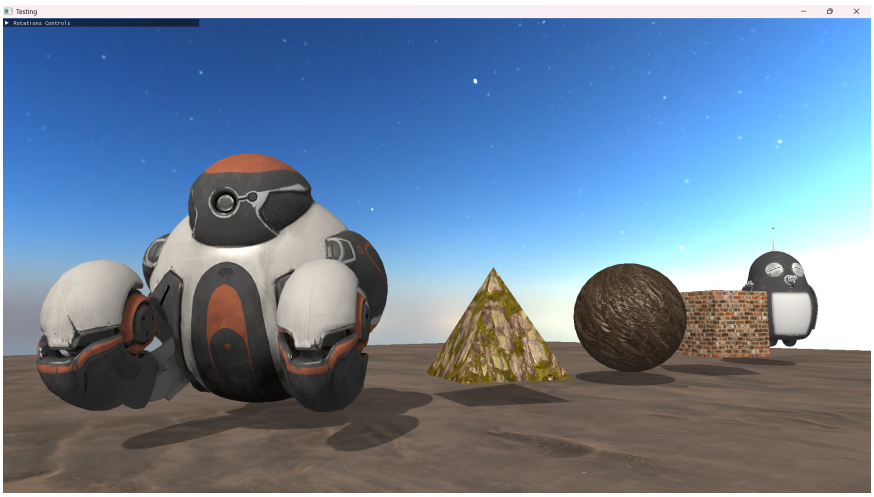


Figure 1: Final renderer output with the stabilised MSM pipeline enabled.

1 Background

Moment Shadow Mapping (MSM) was introduced by Peters and Klein as a filtered shadowing technique that stores several moments of shadow-space depth instead of one raw depth value [11]. Standard shadow mapping stores one depth sample per texel and later compares the current fragment depth against that value. This gives a simple baseline, but it is sensitive to biasing issues and does not respond well when the shadow map is filtered directly.

The key idea of MSM is to replace the single stored depth with a small set of depth moments that give a compact description of the local depth distribution. In practice, the shadow pass writes the moments into an RGBA texture, the texture is filtered, and the scene shader reconstructs a shadow intensity from the filtered data. Compared to ordinary hard depth tests or PCF, MSM is meant to support smoother and more stable shadow transitions while still being practical for real-time rendering [11].

The follow-up 2017 paper, *Improved Moment Shadow Maps*, revisits the original method and focuses on robustness improvements such as better depth conventions, improved bias targets, and more stable handling of filtered moments [12]. In this project, the implementation still mainly follows the 2015 method [11], with only a small number of 2017-inspired additions where they improved stability [12].

For this assignment, the paper background matters for two reasons. First, it shows why MSM is more than "soft shadows with extra blur"; the method is specifically about filtering moment data and reconstructing visibility from it. Second, it gives a clear comparison structure, with hard depth shadows as the baseline, PCF as the common filtered-depth baseline, and MSM as the more advanced research-based method.

2 Implementation Details

2.1 Project Structure and Engine Context

The implementation was built on top of my reusable OpenGL engine [10] rather than as a one-off coursework scene. This mattered for the assignment because the task was not only to reproduce a paper result, but to integrate the technique into an existing real-time renderer in a way that could be demonstrated, compared, and reused. The engine already provided the surrounding systems needed for this, including shader management, geometry and material handling, model loading through Assimp [3], texture loading through stb [15], skybox rendering, and a CMake-based structure that could be reused across projects.

Using that codebase changed the shape of the work. Instead of treating hard shadows, PCF, and MSM as isolated scene-specific features, the shadow work was folded into the renderer itself. In practice, this meant extending the engine-side `ShadowMap` system while keeping the application layer responsible for scene layout, runtime controls, and evaluation captures. This also made the final comparison more meaningful, since all three modes were running inside the same renderer under the same scene and asset setup.

The application layer then provides the scene layout, mode switching, experiment UI, and comparison tooling, with windowing and input handled through GLFW [8], OpenGL loading through GLAD [7], maths through GLM [9], and the comparison UI through Dear ImGui [6].

2.2 Baseline Directional Shadow Mapping

The renderer first implemented a standard directional shadow map. In this baseline path, the scene is rendered from the light's point of view into a depth texture. The visible scene is then rendered from the camera, and each fragment is transformed into light space so that its shadow-space depth can be compared against the stored depth texture.

This gave a working starting point, but it also showed the expected baseline problems. The early version had visible shadow acne and self-shadowing. Before MSM could be compared

fairly, the baseline had to be cleaned up. In practice, this meant making the renderer consistently directional-light based, using the geometric normal rather than the normal-mapped normal for baseline biasing, and exposing the baseline bias values as configurable parameters.

2.3 PCF Depth Shadows

After the baseline depth shadow map was working, PCF was added as the first filtered comparison method. The PCF path still uses the same depth shadow map, but instead of comparing only a single texel, it samples a small neighbourhood around the current coordinate and averages the results. In the current implementation, the PCF radius can be changed at runtime so that quality and cost can be compared as the number of depth comparisons increases. PCF improved the harshness of the edge, but it did not solve acne by itself. In practice, it worked best as a filtered baseline rather than as a full solution to the usual shadow-map artifacts.

2.4 Moment Shadow Mapping

The `ShadowMap` system was then generalized from a depth-only helper into a reusable subsystem with two modes: `Depth` for the baseline hard and PCF paths, and `MSM` for moment shadow mapping. In MSM mode, the shadow pass writes four moments of depth:

$$z,\ z^2,\ z^3,\ z^4$$

These are stored in an RGBA floating-point moment texture. The scene shader later samples this texture and reconstructs a shadow intensity using an MSM reconstruction path based on the 2015 paper structure [11].

2.5 MSM Stabilisation and Practical Improvements

The first MSM implementation worked structurally, but it did not immediately produce a clean final result. The stabilisation work included using rasterized fragment depth when writing moments, adding moment bias for numerical stability, adding receiver bias to reduce self-shadowing, clamping the final reconstructed value, enabling signed-depth support, switching to an improved fallback bias target inspired by the 2017 paper [12], and adding a built-in two-pass Gaussian blur over the moment texture. Not all of these changes produced dramatic visual differences. Some were mainly there for robustness, but together they were important in making MSM stable enough for a fair comparison.

2.6 Scene, UI, and Comparison Workflow

The final scene contains both procedural and imported content. It uses a cube, sphere, and pyramid generated through the custom geometry system, together with a space robot model [14], a penguin model [16], a textured plane using Poly Haven materials [1, 5, 2, 4], and a skybox based on a Poly Haven HDR environment [13]. The application also includes a UI for controlled comparison. Through that UI, it is possible to switch between Hard, PCF, and MSM, change the shadow resolution and frustum settings, adjust the baseline and MSM bias values, vary the PCF radius, and export measurements to CSV for later evaluation, which made

it possible to compare the modes under fixed conditions instead of relying only on casual visual judgement.

3 Results and Evaluation

The evaluation compares hard depth shadows, PCF depth shadows, and MSM with the final stabilisation steps enabled under the same scene, camera, and light setup. All measurements were captured at a shadow resolution of 2048×2048. The renderer exposes the active parameters directly in the UI and writes the averaged metrics to CSV, which made it possible to keep the conditions consistent across captures.

3.1 Performance and Resource Comparison

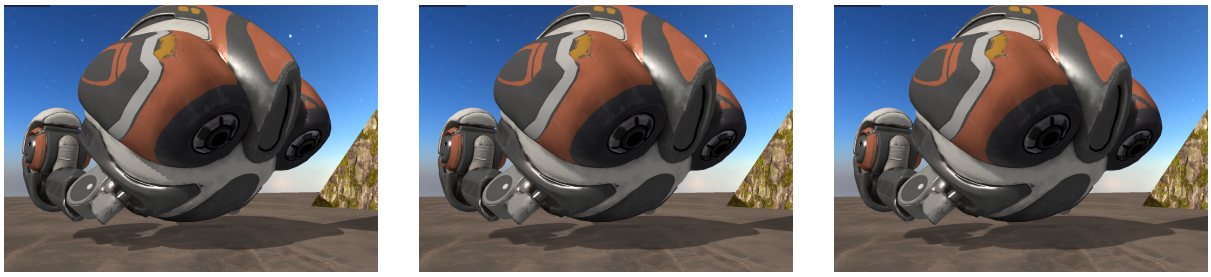
The main recorded metrics were average FPS, average frame time, shadow-pass time, blur-pass time, shadow resolution, and approximate shadow memory usage, which are enough to compare the three modes directly without overloading the evaluation with less useful numbers.

Table 1 shows the main comparison between the baseline Hard mode, the default PCF mode, a wider PCF kernel, and the final MSM mode. The clearest result is that Hard and PCF run at almost identical speed in the current scene, while MSM is substantially more expensive, which is expected because the MSM path uses a four-channel floating-point moment texture, extra reconstruction work in the lighting shader, and a two-pass Gaussian blur.

Table 1: Measured performance and resource comparison across the main shadow modes.

Mode	Resolution	Avg FPS	Avg ms	Shadow ms	Blur ms	Memory (MB)
Hard	2048	164.0	6.114	0.104	–	16
PCF ($r = 1$)	2048	164.1	6.113	0.067	–	16
PCF ($r = 4$)	2048	163.9	6.114	0.099	–	16
MSM (final)	2048	67.1	14.962	0.064	0.011	144

MSM gave the best final visual result, but it also consumed the most memory and had the highest frame cost. PCF stayed close to the baseline in runtime cost, even when the kernel radius was increased from 1 to 4, which shows that the visual gain from MSM does come with a measurable trade-off rather than being a free improvement.



(a) Hard (b) PCF ($r = 1$) (c) MSM

Figure 2: Close-up comparison of Hard, PCF, and MSM.

3.2 Qualitative Comparison

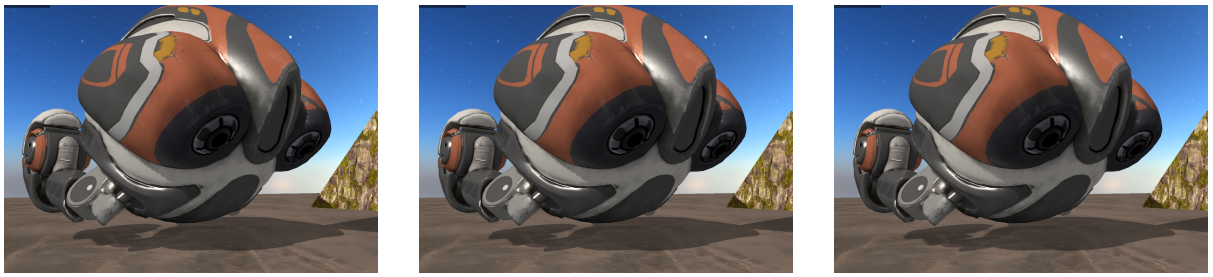
The qualitative comparison focused on shadow edge sharpness, softness of the transition region, visible acne or self-shadowing, light leaking, and stability under motion. Figure 2 shows the clearest visual difference between the three techniques. Hard shadows produce the strictest edge and the clearest depth-test look, but they also come from the baseline path that originally showed the most obvious acne and self-shadowing before cleanup. PCF softens that edge, but still behaves like a filtered version of ordinary depth shadow mapping. In the close-up comparison, the change from Hard to PCF is mainly visible at the shadow boundary, while the change from PCF to MSM is visible in the way the transition becomes smoother without looking washed out. MSM therefore gives the most natural-looking transition while keeping the shadow visually solid. This became much clearer once the stabilisation steps were added.

3.3 MSM Ablation

Table 2: Ablation-style measurements for the MSM pipeline.

Variant	Avg FPS	Avg ms	Blur ms	Signed	Improved Bias
Raw MSM	163.0	6.164	–	Off	Off
No signed depth	71.7	13.968	0.013	Off	On
No improved bias	67.5	14.905	0.013	On	Off
No blur	164.3	6.111	–	On	On
Final MSM	67.1	14.962	0.011	On	On

Table 2 helps separate the visual and runtime roles of the MSM improvements. Disabling blur almost restored the frame time to baseline levels, which shows that the filtered moment map is the main extra cost in the final MSM path. The signed-depth and improved-bias-target changes were much more subtle visually, but they were still worth keeping because they improved stability and made the renderer easier to work with.



(a) Raw (b) No blur (c) Final

Figure 3: MSM ablation: raw, no blur, and final.

The main points from the evaluation are:

- MSM was the most expensive mode in both frame time and shadow memory, but it also gave the strongest visual result after practical stabilisation.
- The Gaussian blur was a necessary part of the final MSM quality, but also an important extra cost.

- PCF remained very competitive in runtime cost, especially at small kernel sizes.

4 Improvements and Limitations

4.1 What Was Improved

Compared to the original baseline, the final renderer includes several improvements that made the comparison stronger:

- a cleaner directional-light pipeline,
- reduced acne in the baseline hard and PCF modes,
- a reusable `ShadowMap` system supporting both depth and MSM resources,
- a functioning MSM path with practical stabilisation,
- Gaussian filtering of the moment map,
- comparison-oriented runtime controls and metric capture support.

4.2 What Remains Limited

The final result is in a good place for the assignment, but it still has clear limits:

- it is not a full paper-complete reproduction of the 2015 method [11],
- the evaluation is useful, but still fairly small in scale,
- the current timing metrics are CPU-side measurements rather than full GPU timing queries,
- some of the 2017 paper was left out, especially the broader treatment of robustness, filtering, and storage [12].

The main limitation is scope. The project reached the point where Hard, PCF, and MSM could be compared fairly inside the same renderer, but it was not pushed into a full reproduction of every idea from both papers. The focus stayed on getting one strong baseline, one standard filtered baseline, and one working MSM implementation that could actually be demonstrated and measured.

4.3 Scope of the 2017 Paper Additions

The 2017 paper was used mainly as a source of practical robustness improvements rather than as a second full implementation target [12]. In this project, the parts that were actually carried over were signed depth, an improved bias target, and a small over-darkening control for suppressing faint light leaking. These changes helped make the MSM path more stable, but they were not dramatic enough to turn this into a separate “Improved MSM” implementation.

Other parts of the 2017 paper [12] were left out. In particular, the project did not attempt the paper’s more complete treatment of improved filtering behaviour, stronger handling of

light leaking, or its work on storage and quantisation. The report also does not claim any of the extended ideas from that paper, such as six-moment extensions or the more advanced translucency-oriented results discussed there. For that reason, the most accurate description of the final result is still a stabilised 2015-style MSM implementation [11] with a small number of 2017-inspired additions.

5 Conclusion

This project achieved a reusable engine implementation of hard shadows, PCF, and a working 2015-style MSM pipeline [11]. The most important outcome was not simply adding MSM, but building a fair comparison by first fixing the baseline shadow setup and then integrating MSM on top of that cleaner foundation.

The implementation matches the core idea of the 2015 paper [11]: storing four moments of depth, filtering the moment data, and reconstructing the final shadow value in the lighting pass. At the same time, the project remains a practical renderer implementation rather than a full paper-complete reproduction. Some of the 2017 paper's robustness ideas [12] were adopted where they improved stability, but the work should still be presented primarily as a stabilised 2015 MSM implementation [11].

Hard shadows provide the baseline, PCF softens edges through repeated depth comparisons, and MSM gives the most natural transition after stabilisation, but at a higher and measurable cost.

References

[1] *Aerial Beach 02*. https://polyhaven.com/a/aerial_beach_02. Beach texture set used for the ground plane from Poly Haven.

[2] *Aerial Rocks 02*. https://polyhaven.com/a/aerial_rocks_02. Rock texture set used on the pyramid from Poly Haven.

[3] *Assimp*. https://github.com/assimp/assimp. Used for imported model loading.

[4] *Bark Willow*. https://polyhaven.com/a/bark_willow. Wood texture set used on the sphere from Poly Haven.

[5] *Brick Wall 005*. https://polyhaven.com/a/brick_wall_005. Brick texture set used on the cube from Poly Haven.

[6] *Dear ImGui*. https://github.com/ocornut/imgui. Immediate-mode GUI library.

[7] *glad*. https://glad.dav1d.de/. Accessed for OpenGL function loading.

[8] *GLFW*. https://www.glfw.org/. Accessed for windowing and input.

[9] *GLM*. https://github.com/g-truc/glm. OpenGL Mathematics library.

[10] Priyansh Nayak. *OpenGL_Engine*. https://github.com/Norged-Out/OpenGL_Engine. Personal reusable OpenGL engine used in this assignment.

[11] Christoph Peters and Reinhard Klein. "Moment Shadow Mapping". In: *Proceedings of the 19th Symposium on Interactive 3D Graphics and Games*. Association for Computing Machinery, 2015, pp. 7–14. DOI: 10.1145/2699276.2699277.

[12] Christoph Peters et al. "Improved Moment Shadow Maps". In: *Journal of Computer Graphics Techniques* 6.1 (2017), pp. 17–67. URL: https://jcgt.org/published/0006/01/03/.

[13] *Qwantani Dusk 2 Pure Sky*. https://polyhaven.com/a/qwantani_dusk_2_puresky. HDR sky environment from Poly Haven.

[14] *Space Maintenance Robot*. https://sketchfab.com/3d-models/space-maintenance-robot-a612ed4feea14fc390beeb3a47943054. Imported space robot model from Sketchfab.

[15] *stb*. https://github.com/nothings/stb. Used for image loading.

[16] *Weathered Penguin Bot*. https://sketchfab.com/3d-models/weathered-penguin-bot-33afc2b450dd4c188b307c82f3b2cbc3. Imported penguin robot model from Sketchfab.